A F T E R L I F E

PROTOCOL & DEVELOPER REFERENCE

custom client development guide · tor hidden service

This document specifies the complete JSON-over-TCP protocol used by the AFTERLIFE server. It covers transport, framing, authentication, every available action with its request/response schema, all error codes, rate limits, pagination, validation rules, and implementation guidance for developers writing custom clients.

TRANSPORT · Plain TCP · loopback 127.0.0.1:2077 · delivered via Tor

FRAMING · Newline-delimited JSON · one request per connection

AUTH · Session token · issued on login · required by every action but four

SERVER · Python 3.10+ · SQLite (WAL) · Fernet encryption at rest

[TABLE OF CONTENTS]

[1] TRANSPORT & FRAMING

The server speaks plain TCP. There is no TLS at the TCP layer — transport security is provided by the Tor hidden service: all traffic between client and server traverses Tor's encrypted onion circuits. Connecting directly to the IP:port without Tor is not supported.

1.1 Connection

Bind address	127.0.0.1 (loopback only, inside the Docker container)
Default port	2077 (configurable via AFTERLIFE_PORT)
Access method	Tor v3 hidden service (.onion address)
Client setup	proxychains or a SOCKS5 library targeting 127.0.0.1:9050
Max connections	100 concurrent (AFTERLIFE_MAX_CONNECTIONS)
Thread pool	32 workers (AFTERLIFE_MAX_WORKERS)
Read timeout	180 seconds per connection

1.2 Framing — One Request Per Connection

Each TCP connection handles exactly one request/response pair. The client sends one JSON object followed by a newline byte (0x0A), reads one JSON object followed by a newline, then the connection closes. Do not pipeline or reuse connections.

```
{"action":"ping"}\n
 -> {"ok":true,"message":"Welcome to AFTERLIFE","data":{"server":"AFTERLIFE","version":1}}\n
```

1.3 Size Limits

Request max	8192 bytes (AFTERLIFE_MAX_REQUEST_LINE_BYTES)
JSON depth	16 levels (AFTERLIFE_MAX_JSON_DEPTH)
Response max	No server-imposed limit; clients should cap reads at 1 MiB

> **NOTE** Requests exceeding the byte limit are rejected and the connection is closed. The largest user payload is a forum thread body (2096 chars); even fully newline-escaped it fits well within 8192 bytes.

[2] REQUEST ENVELOPE

Every request is a flat JSON object. The action field is mandatory; all other fields are action-specific. The top level must be a JSON object — arrays and primitives are rejected with bad_request.

2.1 Mandatory Field

action	string	Name of the operation. Must be one of the valid action strings.

2.2 Session Token

All actions except ping, get_challenge, register, and login require an authenticated session. There is no anonymous access: list_jobs, job_details, list_threads, thread_details, and search_threads all require a valid session_token. Pass the token returned by login as session_token in every authenticated request.

session_token	string	Opaque token issued by login. Required by all actions except ping/register/login.

```
{"action":"profile","session_token":"AbC...xyz"}
```

2.3 Pagination Request Field

The list_jobs, list_threads, and search_threads actions accept an optional page field. It is a 1-based page index; values below 1 default to 1, and values past the last page clamp to the last page. Page size is fixed server-side at 10 and is not client-adjustable. Omitting page returns page 1.

page	integer	Optional 1-based page number. Default 1. Clamped to [1, total_pages].

2.4 Proof-of-Work Fields

register, login, and rate_user additionally require a solved proof-of-work challenge (Section 7.0). Obtain one with get_challenge, solve it, and attach both fields. The challenge is consumed on use, so a fresh solve is required for every protected request — there is no reusable token.

challenge_id	string	ID of a challenge issued by get_challenge for this action's purpose.
nonce	string	The solving nonce. sha256(prefix:nonce) must meet the required difficulty.

[3] RESPONSE ENVELOPE

Every response is a JSON object with a consistent top-level shape regardless of the action.

3.1 Envelope Fields

FIELD	TYPE	NULL	DESCRIPTION
ok	bool	no	true on success, false on any error.
message	string	no	Human-readable result string. For display only — never parse it for logic.
data	object	yes	Action-specific payload. An object when present; {} when there is no data.
error	string	yes	Machine-readable error code. Present only when ok is false. See Section 4.
retry_after	int	yes	Seconds to wait before retrying. Present on rate_limited and login_throttled.

3.2 Success and Error Shapes

```
{"ok":true,"message":"...","data":{ ... }}
{"ok":false,"message":"...","error":"error_code"}
{"ok":false,"message":"...","error":"rate_limited","retry_after":7}
```

3.3 Pagination Object

Responses from list_jobs, list_threads, and search_threads include a pagination object alongside the items array. Use it to drive page navigation.

FIELD	TYPE	NULL	DESCRIPTION
page	int	no	Current (clamped) page number, 1-based.
per_page	int	no	Items per page. Always 10.
total	int	no	Total matching items (search: capped at 50 collected matches).
total_pages	int	no	Total number of pages (at least 1).
has_prev	bool	no	Whether a previous page exists.
has_next	bool	no	Whether a next page exists.

```
{"ok":true,"message":"OK","data":{"jobs":[ ... ],
 "pagination":{"page":1,"per_page":10,"total":23,"total_pages":3,"has_prev":false,"has_next":true}}}
```

[4] ERROR CODES

The error field in failed responses contains one of the following strings. Never parse the message field for logic — it is for display only and may change.

ERROR CODE	MEANING
bad_request	Malformed JSON, wrong top-level type, depth exceeded, or missing/invalid action.
unknown_action	The action string is not in the set of valid actions.
validation_error	A field failed a validation rule (length, regex, type, range).
auth_required	The action requires a session but none was provided, or the token is invalid/expired.
login_failed	Incorrect nickname or password, or the account is banned.
login_throttled	Too many failed logins for this nickname from this address. retry_after is set.
register_failed	Nickname already taken.
challenge_failed	Proof-of-work challenge missing, expired, wrong purpose, already used, or unsolved.
not_found	The requested job, chat, message, or thread does not exist or is not visible to you.
rate_limited	A rate limiter tripped (global, parse, message, session, forum-write, or search). retry_after set.
accept_failed	Could not accept the job (already accepted, not open, blocked, bad token).
withdraw_failed	Could not withdraw (not accepted, or you are the selected worker).
select_failed	Could not select worker (not in pool, banned, blocked).
status_failed	Status transition not permitted (e.g. reopening a done contract).
delete_failed	Job, thread, or comment not found, or caller is not admin.
ban_failed	Cannot ban the target (admin, already banned, not found).
wipe_failed	Cannot wipe the target (admin, not found, or caller is not admin).
rating_failed	Rating not permitted (self, blocked, cooldown, same rating).
block_failed	Block not permitted (self, admin, already blocked).
unblock_failed	User was not blocked.
thread_failed	Thread could not be created (banned, or reputation -10 or below).
comment_failed	Comment could not be posted (banned, reputation gate, blocked, thread missing).
chat_failed	Cannot open chat (blocked, banned, self).
messages_failed	Chat not found, or you are not a participant.
message_failed	Message not found.
send_failed	Message could not be sent (blocked, chat not found).
server_busy	Connection limit reached. Reconnect after a delay.
server_error	Unexpected internal server error.

[5] RATE LIMITS

The server enforces several independent limiters. All Tor clients share the source IP 127.0.0.1, so IP-keyed limiters apply collectively. The session, message, forum-write, and search limiters are keyed on the session token, giving each user an individual bucket — these are what actually throttle an individual user.

FIELD	TYPE	NULL	DESCRIPTION
Global	per IP	—	500 requests / 10 s. Shared by all Tor clients. Returns rate_limited.
Login throttle	IP+nick	—	5 failed logins / 300 s locks that nickname from the IP for 300 s.
Parse errors	per IP	—	50 JSON parse errors / 10 s, then rate_limited.
Session	session	—	240 requests / 60 s per token. Applies to every authenticated action.
Messages	session	—	10 messages / 30 s per token (send_message).
Forum write	session	—	10 / 60 s per token (create_thread and post_comment combined).
Search	session	—	10 / 60 s per token (search_threads).

NOTE When retry_after is present, wait that many seconds before retrying the same action. An over-length password at login is rejected before hashing, and is counted as a failed attempt for the login throttle.

NOTE In addition to these limiters, register, login, and rate_user require a client-solved proof-of-work challenge (Section 7.0) — a CPU cost imposed on every attempt to slow automated abuse.

[6] VALIDATION RULES

All fields are validated server-side. A validation_error response includes a human-readable message describing the failure. This table is definitive.

FIELD	TYPE	NULL	DESCRIPTION
nickname	string	—	3-12 chars. Regex ^[A-Za-z0-9_]{3,12}$.
password	string	—	8-64 chars. Over-length is rejected at login before hashing.
title	string	—	1-32 chars. Base text charset; no forbidden chars.
description	string	—	1-256 chars. Base text charset.
message	string	—	1-128 chars. Base text charset, single line.
thread title	string	—	1-100 chars. Base text charset, single line.
thread body	string	—	1-2096 chars and <= 4096 bytes. Newlines allowed; text only.
comment	string	—	1-1000 chars. Newlines allowed; text only.
query	string	—	3-64 chars (search_threads). Minimum 3 enforced before any decrypt.
reward	string	—	Digits only, parsed as int. Range 1 to 99,999,999.
min_reputation	string	—	Integer string (may be negative). Range -999,999 to 999,999.
status	string	—	One of: open, done, cancelled (lowercase).
rating	string	—	One of: positive, negative (lowercase).
is_private	bool	—	JSON boolean. Strings are not accepted.
page	int	—	Coerced via int(); non-numeric defaults to 1; clamped to valid range.
ids	int	—	job_id, worker_id, chat_id, message_id, thread_id, comment_id: positive ints.

NOTE Base text charset: [A-Za-z0-9 _.,:;!?()-[]@]. It is ASCII-only, which inherently excludes emoji and binary/image data. Thread bodies and comments additionally permit newlines.

NOTE Forbidden characters in all text fields: ' " \ / % + (single quote, double quote, backslash, forward slash, percent, plus).

NOTE min_reputation is silently capped to min(supplied, author_reputation) at creation — you cannot require a higher reputation than you hold.

[7] AUTHENTICATION

7.0 Proof-of-Work Challenge

register, login, and rate_user are gated by a proof-of-work challenge. The flow is: (1) call get_challenge with the target purpose; (2) brute-force a nonce so that sha256("prefix:nonce") has at least difficulty leading zero bits; (3) submit the action with challenge_id and nonce. Verification is a single hash; solving costs about 2^difficulty hashes. Challenges are one-time and short-lived, so each protected action requires its own fresh solve.

get_challenge

NO AUTH Issue a one-time proof-of-work challenge.

Request fields

purpose	string	One of: register, login, rate_user.

Response data fields

FIELD	TYPE	NULL	DESCRIPTION
challenge_id	string	no	Opaque ID. Submit it with the solved action.
prefix	string	no	Random hex string to hash with your nonce.
difficulty	int	no	Required leading zero bits of sha256(prefix:nonce).
algorithm	string	no	Always 'sha256-leading-zero-bits'.
expires_in	int	no	Seconds until the challenge expires (default 180).

NOTE Default difficulty is 20 bits (~1 s average in a pure-Python client), tunable via AFTERLIFE_POW_DIFFICULTY. Challenges expire after ~180 s and are consumed on first valid use; replays return challenge_failed.

```
{"action":"get_challenge","purpose":"login"}
-> {"ok":true,"message":"Solve the proof-of-work challenge.","data":{
 "challenge_id":"a1b2...","prefix":"9f3c...","difficulty":20,
 "algorithm":"sha256-leading-zero-bits","expires_in":180}}
```

register

NO AUTH Create a new user account. Requires proof-of-work.

Request fields

nickname	string	3-12 chars, letters/digits/underscore.
password	string	8-64 chars.
challenge_id	string	From get_challenge with purpose=register.
nonce	string	Solving nonce for the challenge.

NOTE Returns an empty data object on success. No session is created — call login after registering.

```
{"action":"register","nickname":"corvo","password":"hunter2hunter2"}
-> {"ok":true,"message":"User created.","data":{}}
```

login

NO AUTH Authenticate and receive a session token. Requires proof-of-work.

Request fields

nickname	string	Account nickname.
password	string	Account password (rejected if longer than 64 chars).
challenge_id	string	From get_challenge with purpose=login.
nonce	string	Solving nonce for the challenge.

Response data fields

FIELD	TYPE	NULL	DESCRIPTION
session_token	string	no	Opaque token. Include in all subsequent authenticated requests.
nickname	string	no	Confirmed nickname.
reputation	int	no	Current reputation score.
is_admin	bool	no	Whether the account has admin privileges.
is_banned	bool	no	Always false on success (banned users cannot log in).

> **NOTE** A successful login invalidates all existing sessions for the same user — only one active session per account. Each failed login adds a throttle strike and incurs a 1-second delay. The proof-of-work is consumed on every attempt, successful or not, to slow credential stuffing.

```
{"action":"login","nickname":"corvo","password":"hunter2hunter2"}
-> {"ok":true,"message":"Login successful.","data":{"session_token":"AbC...xyz",
 "nickname":"corvo","reputation":3,"is_admin":false,"is_banned":false}}
```

logout

AUTH REQUIRED Invalidate the current session.

> **NOTE** Idempotent — passing an invalid or expired token still returns ok:true.

```
{"action":"logout","session_token":"AbC...xyz"} -> {"ok":true,"message":"Logged out.","data":{}}
```

[8] USER & PROFILE

profile

AUTH REQUIRED Retrieve the authenticated user's own profile.

Response data fields

FIELD	TYPE	NULL	DESCRIPTION
nickname	string	no	Account nickname.
reputation	int	no	Current reputation score. Can be negative.
created_at	int	no	Unix timestamp of account creation.
is_admin	bool	no	Admin privilege flag.
is_banned	bool	no	Ban status. Always false for an active session.

```
{"action":"profile","session_token":"..."}
-> {"ok":true,"message":"OK","data":{"nickname":"corvo","reputation":3,
 "created_at":1713430800,"is_admin":false,"is_banned":false}}
```

Reputation starts at 0. It increases by +1 when a contract you are selected for is marked done, and by the ratings of other users (+1 positive, -1 negative). A reputation of -10 or below blocks forum posting and commenting.

[9] JOB BOARD

list_jobs

AUTH REQUIRED List contracts, newest first. Paginated, block-filtered.

Request fields

status	string	Optional. One of open, done, cancelled. Omit for all statuses.
page	integer	Optional 1-based page. Default 1.

Response data fields

FIELD	TYPE	NULL	DESCRIPTION
jobs	array	no	Up to 10 Job Summary objects (Section 15).
pagination	object	no	Pagination object (Section 3.3).

> **NOTE** Contracts authored by users in a block relation with you (either direction) are excluded from both the page and the total count.

```
{"action":"list_jobs","status":"open","page":2,"session_token":"..."}
```

my_jobs

AUTH REQUIRED List all contracts authored by the caller. Not paginated.

Response data fields

FIELD	TYPE	NULL	DESCRIPTION
jobs	array	no	Array of Job Author objects (no author fields).

my_accepts

AUTH REQUIRED List all contracts the caller has accepted. Not paginated.

Response data fields

FIELD	TYPE	NULL	DESCRIPTION
jobs	array	no	Array of Job Summary objects; description is null in this view.

[10] JOB LIFECYCLE

create_job

AUTH REQUIRED Post a new contract.

Request fields

title	string	1-32 chars.
description	string	1-256 chars.
reward	string	Numeric string, 1 to 99,999,999.
min_reputation	string	Integer string. Capped to author reputation.
is_private	bool	true to generate a private token; false for a public contract.

Response data fields

FIELD	TYPE	NULL	DESCRIPTION
job_id	int	no	ID of the newly created contract.
private_token	string	yes	One-time unlock/accept token. Only when is_private is true; empty otherwise.

> **NOTE** The private_token is shown exactly once and stored only as a PBKDF2-SHA256 hash. There is no recovery endpoint.

job_details

AUTH REQUIRED Full details for one contract.

Request fields

job_id	integer	Required contract ID.
unlock_token	string	Optional. Private token to unlock a private contract's description.

Returns a Job Detail object (Section 15) including viewer_reputation, description_visible, the decrypted description (when visible), viewer_has_accepted, is_author, is_admin, selected_worker_id, and worker_pool (author/admin only).

accept_job

AUTH REQUIRED Apply for a contract. Auto-creates a chat with the author.

Request fields

job_id	integer	Required.
private_token	string	Required for private contracts; must match the stored hash.

withdraw_job

AUTH REQUIRED Withdraw a previously submitted application.

job_id	integer	Required. Cannot withdraw once selected as worker.

select_worker

AUTH REQUIRED Assign an accepted worker. Author/admin only.

job_id	integer	Required.
worker_id	integer	Required. Must be a user_id in the worker pool.

set_status

AUTH REQUIRED Change contract status. Author/admin only.

job_id	integer	Required.
status	string	One of open, done, cancelled.

> **NOTE** done requires a selected worker, is terminal (cannot be reopened or cancelled), and grants the selected worker +1 reputation.

[11] MESSAGING

open_chat

AUTH REQUIRED Open or create a 1:1 chat with another user.

Request fields

nickname	string	Target user's nickname.

Response data fields

FIELD	TYPE	NULL	DESCRIPTION
chat_id	int	no	ID of the chat session.
other_user_id	int	no	Internal ID of the other participant.
other_nickname	string	no	Nickname of the other participant.
created_at	int	no	Unix timestamp of chat creation.
updated_at	int	no	Unix timestamp of last message.
last_message	string	yes	Decrypted body of the most recent message.
unread_count	int	no	Unread messages for the caller.

> **NOTE** Cannot open a chat with yourself, a banned user, or anyone in a block relation with you. A new chat inserts a 'started a conversation' system message.

list_chats

AUTH REQUIRED List the caller's chats, most-recently-updated first.

Response data fields

FIELD	TYPE	NULL	DESCRIPTION
chats	array	no	Array of Chat Summary objects (Section 15).

list_messages

AUTH REQUIRED All messages in a chat. Marks unread as read.

Request fields

chat_id	integer	Required.

Response data fields

FIELD	TYPE	NULL	DESCRIPTION
chat_id	int	no	The chat ID.
other_nickname	string	no	Nickname of the other participant.
messages	array	no	Array of Message objects, ordered by created_at ascending.

> **NOTE** Calling list_messages marks every unread message in the chat as read for the caller — the primary read mechanism.

read_message

AUTH REQUIRED Retrieve a single message by ID; marks it read.

message_id	integer	Required.

Returns the Message object fields (id, chat_id, sender_id, sender_nickname, message_type, body, created_at). Retained in the protocol; the bundled client now reads whole conversations via list_messages instead.

send_message

AUTH REQUIRED Send a message in a chat.

Request fields

chat_id	integer	Required.
message	string	1-128 chars. Subject to message validation.

Response data fields

FIELD	TYPE	NULL	DESCRIPTION
message_id	int	no	ID of the newly created message.

> **NOTE** Rate limited at 10 messages / 30 s per session token. Cannot send to anyone in a block relation with you.

[12] FORUM

A single flat board of text threads with comments. Titles, bodies, and comments are Fernet-encrypted at rest. Posting requires reputation above -10; reading and searching do not. Block isolation applies: threads and replies from users in a block relation with you are hidden.

create_thread

AUTH REQUIRED Start a new thread. Forum-write rate limited.

Request fields

title	string	1-100 chars, single line.
body	string	1-2096 chars, <= 4096 bytes. Newlines allowed.

Response data fields

FIELD	TYPE	NULL	DESCRIPTION
thread_id	int	no	ID of the new thread.

```
{"action":"create_thread","title":"Escrow tips","body":"Line one.\nLine two.","session_token":"..."}
-> {"ok":true,"message":"Thread created.","data":{"thread_id":12}}
```

list_threads

AUTH REQUIRED List threads, newest activity first. Paginated, block-filtered.

Request fields

page	integer	Optional 1-based page. Default 1.

Response data fields

FIELD	TYPE	NULL	DESCRIPTION
threads	array	no	Up to 10 Thread Summary objects (Section 15).
pagination	object	no	Pagination object (Section 3.3).

thread_details

AUTH REQUIRED One thread with its opening post and replies.

Request fields

thread_id	integer	Required.

Returns a Thread Detail object (Section 15): the Thread Summary fields plus body, is_author, is_admin, and posts (an array of Post objects ordered oldest-first). Replies from users you have blocked are omitted; admins see everything.

post_comment

AUTH REQUIRED Reply to a thread. Forum-write rate limited.

Request fields

thread_id	integer	Required.
body	string	1-1000 chars. Newlines allowed.

Response data fields

FIELD	TYPE	NULL	DESCRIPTION
comment_id	int	no	ID of the new comment.

search_threads

AUTH REQUIRED Substring search over titles and bodies. Paginated.

Request fields

query	string	3-64 chars. The 3-char minimum is enforced before any decryption.
page	integer	Optional 1-based page. Default 1.

Response data fields

FIELD	TYPE	NULL	DESCRIPTION
threads	array	no	Up to 10 Thread Summary objects for the page.
query	string	no	Echo of the search query.
pagination	object	no	Pagination object; total is capped at 50 collected matches.

> NOTE Search decrypts candidate rows in memory (ciphertext is non-deterministic, so SQL LIKE cannot be used). The 3-char minimum plus the per-session search limiter (10 / 60 s) bound this cost.

delete_thread

AUTH REQUIRED Delete a thread and its comments. Admin only.

thread_id	integer	Required.

delete_comment

AUTH REQUIRED Delete a single comment by ID. Admin only.

comment_id	integer	Required.

[13] SOCIAL GRAPH

rate_user

AUTH REQUIRED Set a positive or negative rating. Requires proof-of-work.

Request fields

nickname	string	Target user's nickname.
rating	string	'positive' or 'negative'.
challenge_id	string	From get_challenge with purpose=rate_user.
nonce	string	Solving nonce for the challenge.

> **NOTE** Cannot rate yourself, a banned user, or anyone in a block relation. A rating can be changed at most once per 24 hours. Setting the same rating again returns rating_failed. positive = +1, negative = -1; flipping applies a delta of 2.

> **NOTE** If a user accumulates more than 5 currently-negative ratings within 24 hours, the server writes a reputation_negative_burst warning to the audit log (brigade detection). This is server-side only; no field is returned to clients.

block_user

AUTH REQUIRED Block a user. Prevents all interaction both ways.

nickname	string	User to block. Admins cannot be blocked; cannot block self.

unblock_user

AUTH REQUIRED Remove a block.

nickname	string	User to unblock. unblock_failed if not blocked.

list_blocks

AUTH REQUIRED List users blocked by the caller.

Response data fields

FIELD	TYPE	NULL	DESCRIPTION
blocks	array	no	Array of Block Entry objects (Section 15).

[14] ADMIN ACTIONS

These actions require is_admin on the authenticated session. Non-admin callers receive the corresponding *_failed error. Every moderation action is recorded in the server audit log with the actor, target, and affected IDs.

ban_user

AUTH REQUIRED Permanently ban a user. Admin only.

nickname	string	User to ban.

> **NOTE** Sets is_banned, deletes the user's job accepts, clears selected_worker_id on their assigned open jobs, and invalidates their sessions. Existing content remains. Admins cannot be banned. There is no unban action.

wipe_user

AUTH REQUIRED Ban and purge a user's content. Admin only.

nickname	string	User to wipe.

> **NOTE** Bans the account (keeping the account row) and deletes all of its jobs, threads (cascading their comments), and comments on other threads. Chats and the account row are preserved; sessions are invalidated. The message reports the removed counts. Admins cannot be wiped.

delete_job

AUTH REQUIRED Permanently delete a contract and its accepts. Admin only.

job_id	integer	Required. Deletion is permanent and immediate.

[15] DATA TYPE REFERENCE

Job Summary Object (list_jobs, my_jobs, my_accepts)

FIELD	TYPE	NULL	DESCRIPTION
id	int	no	Contract ID.
title	string	no	Decrypted title.
reward	int	no	Reward amount.
min_reputation	int	no	Minimum reputation required.
is_private	bool	no	Private contract flag.
status	string	no	open, done, or cancelled.
author_id	int	no	Author user ID.
author_nickname	string	no	Author nickname.
author_is_banned	bool	no	Whether the author is banned.
author_display	string	no	Nickname with [banned] prefix if banned.
accept_count	int	no	Number of accepted applications.
created_at / updated_at	int	no	Unix timestamps.
viewer_reputation	int	no	Caller's reputation (always present — calls are authenticated).
not_enough_reputation	bool	no	true if the caller cannot accept due to reputation.

Thread Summary Object (list_threads, search_threads)

FIELD	TYPE	NULL	DESCRIPTION
id	int	no	Thread ID.
title	string	no	Decrypted title.
author_id	int	no	Author user ID.
author_nickname	string	no	Author nickname.
author_is_banned	bool	no	Whether the author is banned.
author_display	string	no	Nickname with [banned] prefix if banned.
reply_count	int	no	Number of comments on the thread.
created_at / updated_at	int	no	Creation and last-activity timestamps.

Thread Detail Object (thread_details) — adds to Thread Summary

FIELD	TYPE	NULL	DESCRIPTION
body	string	no	Decrypted opening-post body.
is_author	bool	no	Whether the caller authored the thread.
is_admin	bool	no	Whether the caller is an admin.
posts	array	no	Array of Post objects, oldest first (blocked authors omitted).

Post Object (thread_details.posts)

FIELD	TYPE	NULL	DESCRIPTION
id	int	no	Comment ID.
author_id	int	yes	Author user ID; null if the author was deleted.
author_nickname	string	no	Author nickname, or '[deleted user]'.
author_display	string	no	Nickname with [banned] prefix if banned.
body	string	no	Decrypted comment body.
created_at	int	no	Unix timestamp.

Challenge Object (get_challenge)

FIELD	TYPE	NULL	DESCRIPTION
challenge_id	string	no	Opaque challenge ID to submit with the action.
prefix	string	no	Random hex prefix to hash with the nonce.
difficulty	int	no	Required leading zero bits of sha256(prefix:nonce).
algorithm	string	no	'sha256-leading-zero-bits'.
expires_in	int	no	Seconds until expiry.

Pagination Object (list_jobs, list_threads, search_threads)

FIELD	TYPE	NULL	DESCRIPTION
page	int	no	Current page (clamped, 1-based).
per_page	int	no	Items per page (always 10).
total	int	no	Total matching items.
total_pages	int	no	Total pages (>= 1).
has_prev / has_next	bool	no	Whether adjacent pages exist.

Chat Summary Object (list_chats, open_chat)

FIELD	TYPE	NULL	DESCRIPTION
chat_id	int	no	Chat ID.
other_user_id	int	no	Other participant's user ID.
other_nickname	string	no	Other participant's nickname.
created_at / updated_at	int	no	Creation and last-message timestamps.
last_message	string	yes	Decrypted body of the most recent message.
last_message_type	string	yes	'user' or 'system'.
last_message_at	int	yes	Timestamp of the most recent message.
unread_count	int	no	Unread messages for the caller.

Message Object (list_messages.messages)

FIELD	TYPE	NULL	DESCRIPTION
id	int	no	Message ID.
chat_id	int	no	Parent chat ID.

FIELD	TYPE	NULL	DESCRIPTION
sender_id	int	yes	Sender user ID; null for system messages.
sender_nickname	string	no	Sender nickname, or 'system'.
message_type	string	no	'user' or 'system'.
body	string	no	Decrypted message body.
created_at	int	no	Unix timestamp.
is_read	bool	no	Whether the viewer has read this message.

Block Entry Object (list_blocks.blocks)

FIELD	TYPE	NULL	DESCRIPTION
id	int	no	Blocked user's ID.
nickname	string	no	Blocked user's nickname.
reputation	int	no	Blocked user's reputation.
is_banned	bool	no	Whether the blocked user is also banned.
created_at	int	no	When the block was created.

[16] IMPLEMENTATION NOTES

16.1 One Connection Per Request

Open a fresh TCP connection for every request. The server closes the connection after sending the response. Connection reuse and pipelining are not supported.

16.2 Connecting Through Tor

proxychains	Wrap the whole process. Transparent, but flaky with some Python socket setups.
SOCKS5 library	PySocks: socks.socksocket().set_proxy(SOCKS5,'127.0.0.1',9050). More reliable.

> **NOTE** Tor adds latency. Use a socket timeout of at least 60 s. A freshly started hidden service can take 30-120 s to become reachable while the descriptor publishes.

16.3 Minimum Client Checklist

Framing	Append \n (0x0A) to each request; read until \n in the response.
Encoding	UTF-8 both ways: json.dumps(...).encode('utf-8').
Session	Store session_token after login; inject into every authenticated request.
Reconnect	Fresh TCP connection per request; do not keep-alive.
Auth	All actions but ping/get_challenge/register/login need a token — no anonymous mode.
Proof-of-work	For register/login/rate_user: get_challenge, solve, attach challenge_id+nonce.
Pagination	Send page; read pagination.has_next / has_prev to drive navigation.
retry_after	Respect it; back off before retrying rate-limited requests.
Error branch	Always check ok before reading data. Never assume data is present.
Response cap	Cap reads at 1 MiB to guard against a hostile or malformed server.

16.4 Session Lifecycle

Sessions expire after 1 hour of inactivity (SESSION_IDLE_SECONDS = 3600). Any valid request resets the idle timer. Detect expiry by checking for auth_required in the error field and re-authenticate.

16.5 Encryption Notes

Titles, descriptions, message bodies, and all forum thread/comment text are encrypted at rest with Fernet (AES-128-CBC + HMAC-SHA256). The server returns plaintext in responses, so clients implement no decryption. Passwords and private tokens are stored as PBKDF2-SHA256 hashes with 200,000 iterations.

> **NOTE** This reference reflects the protocol as implemented. Server constants (rate limits, sizes, timeouts) can be overridden via environment variables — treat the values here as defaults, not guarantees.

AFTERLIFE · protocol & developer reference · end of document